

Date	15 March 2026
Team ID	XXXXXX
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Testing Tool Used	Apache JMeter
Type of Testing	Load Testing & Stress Testing
Target Module / API	Backend ML Prediction API Endpoint (`/api/v1/predict`)
Test Environment	Cloud Hosted Server (AWS EC2)
Test Date	25 March 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Normal Load: Standard daily banking traffic.	50	60	System processes all predictions seamlessly without queueing.
2	Peak Load: High traffic during a credit card promotion.	200	120	Response time stays under 2 seconds; zero dropped requests.
3	Stress Test: Extreme unexpected spike.	500	60	System starts queueing but does not crash or corrupt data.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.8 seconds	Pass	XGBoost inference is fast.
2	Response Time (Max)	< 5 seconds	2.1 seconds	Pass	Minor delay during stress test.
3	Throughput (Req/sec)	> 100 Req/sec	150 Req/sec	Pass	FastAPI async handles concurrent loads efficiently.
4	Error Rate	< 1%	0.2%	Pass	Negligible timeout errors during peak.
5	CPU Utilization	< 80%	65%	Pass	Well within EC2 instance limits.
6	Memory Utilization	< 80%	70%	Pass	Fits comfortably inside 16GB RAM allocation.

Step 4: Observations & Analysis

Key Findings:

The system successfully handles up to 200 concurrent virtual users with sub-second response times. The combination of FastAPI's asynchronous routing and optimized XGBoost inference allows for high throughput during peak load conditions.

Bottlenecks Identified:

The SHAP (Explainable AI) value generation calculation proved to be CPU-intensive. During the 500-user stress test, this specific calculation caused minor latency spikes (max 2.1 seconds).

Optimization Steps Taken:

To resolve the SHAP bottleneck, we cached the frequent background datasets used for explainability in memory and increased the Gunicorn worker count to better parallelize the blocking ML calculations.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

[Insert Screenshot 1 here - e.g., JMeter Response Time Graph]

[Insert Screenshot 2 here - e.g., JMeter Throughput / Error Rate Table]